

Lecture 23: Variational Autoencoders

Give the latent space no holes, then sample it

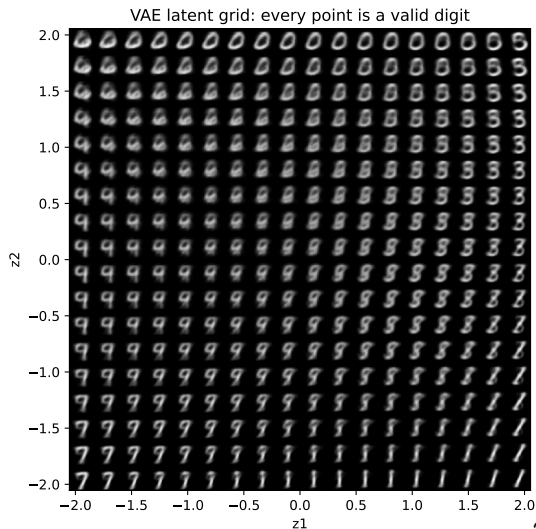
Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

What if every point were a digit?

Last lecture ended in failure: the plain autoencoder could not **generate** - random codes decode to blurry blobs.

But look what we *want* (right): a latent space where **every point decodes to a valid digit**, and neighbours morph smoothly into one another.

If the whole space were like this, a **random draw** would produce a brand-new digit. That is a **generative model**. This lecture builds it.



Outline

From autoencoder to generative model

The VAE objective: the ELBO

The reparameterization trick

The VAE in action

Honest limits and the road ahead

From autoencoder to generative model

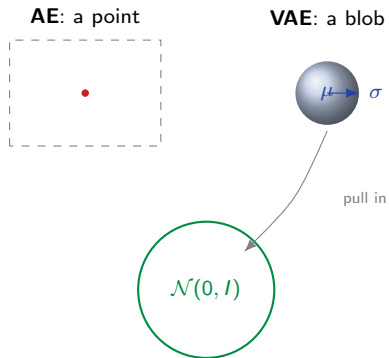
An autoencoder memorizes points. We want a space we can sample.

The fix: a blob, not a point

A plain autoencoder maps each input to a single **point** $\mathbf{z}$. The gaps between points are undefined, so a random draw lands in a hole.

The fix, in one line: make the encoder output a little **Gaussian blob** per input - a mean $\mu(\mathbf{x})$ and a spread $\sigma(\mathbf{x})$ - and push all the blobs to fill a standard normal $\mathcal{N}(0, I)$ with no gaps.

Two new ingredients: **(1)** the encoder predicts μ, σ ; **(2)** a regularizer pulls every blob toward $\mathcal{N}(0, I)$. The next two sections make each precise.



The VAE objective: the ELBO

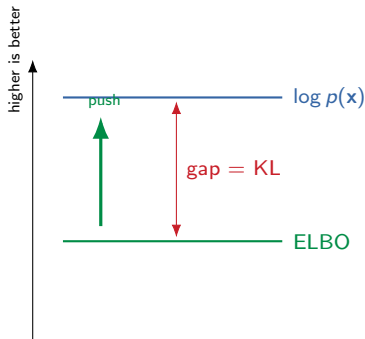
Where the loss comes from - honestly.

The plan, before the math

We want the training images to be **likely**: maximize $\log p(\mathbf{x})$. But that means adding up *every* possible code that could have made $\mathbf{x}$ - we cannot compute it.

The move: build a quantity we *can* compute - the **ELBO** - that never rises above $\log p(\mathbf{x})$. Push the ELBO up, and $\log p(\mathbf{x})$ is dragged up with it.

The **gap** between floor and ceiling is exactly how wrong the encoder's guess is. Raising the ELBO does two jobs at once: fit the data better *and* make the encoder a better guesser. ($\text{KL}(a \parallel b)$ just measures how far apart two distributions are - zero when they match.)



The loss is then a **tug of war**: reconstruction spreads codes apart, the KL tidies them toward $\mathcal{N}(0, I)$.

A latent-variable model

Assume each image is generated by first drawing a latent code, then decoding it:

$$p(\mathbf{x}) = \int p(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z}, \quad p(\mathbf{z}) = \mathcal{N}(0, I),$$

where the decoder $p(\mathbf{x} | \mathbf{z})$ is a neural network.

We would like to train by maximizing $\log p(\mathbf{x})$. But that integral runs over every possible code $\mathbf{z}$ - it is **intractable**. We cannot compute it, let alone differentiate it.

The way out: introduce the **encoder** $q(\mathbf{z} | \mathbf{x})$ as a cheap stand-in for the true (unknown) posterior $p(\mathbf{z} | \mathbf{x})$.

Deriving the ELBO

Start from $\log p(\mathbf{x})$, multiply inside by $\frac{q(\mathbf{z}|\mathbf{x})}{q(\mathbf{z}|\mathbf{x})}$, and split the logarithm:

$$\begin{aligned}\log p(\mathbf{x}) &= \underbrace{\mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x} | \mathbf{z})] - \text{KL}(q(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z}))}_{\text{ELBO}(\mathbf{x})} \\ &\quad + \underbrace{\text{KL}(q(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z} | \mathbf{x}))}_{\geq 0}.\end{aligned}$$

The last term is a KL divergence, so it is ≥ 0 . Drop it and you get a **lower bound**:

$$\log p(\mathbf{x}) \geq \text{ELBO}(\mathbf{x}).$$

We cannot maximize $\log p(\mathbf{x})$, so we **maximize the ELBO** instead (the *Evidence Lower Bound*).

Reading the two terms

$$\text{ELBO}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x} | \mathbf{z})]}_{\text{reconstruction}} - \underbrace{\text{KL}(q(\mathbf{z} | \mathbf{x}) \| \mathcal{N}(0, I))}_{\text{regularizer}}$$

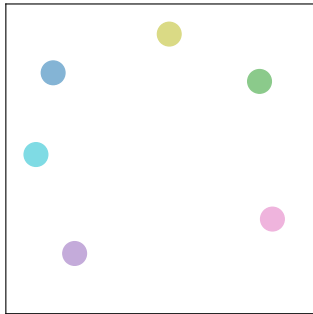
- **Reconstruction:** decode $\mathbf{z}$ and rebuild $\mathbf{x}$ well - this is exactly the autoencoder loss from L22.
- **KL regularizer:** keep each blob $q(\mathbf{z} | \mathbf{x})$ close to $\mathcal{N}(0, I)$ - this is what fills the latent space with no holes.

Callback: the KL term is a **prior** on the code - a regularizer, the same role weight decay played in ch5. Closed form for two Gaussians: $\text{KL} = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - \ln \sigma_j^2 - 1)$, smallest when $\mu = 0, \sigma = 1$ - i.e. the blob *is* the prior.

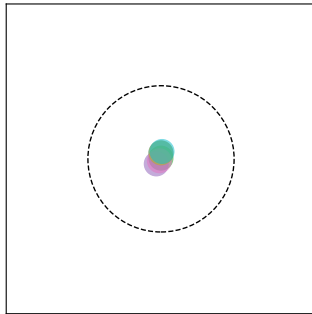
The tug of war

The two ELBO terms pull in opposite directions - their **balance** is what makes the latent both faithful and samplable:

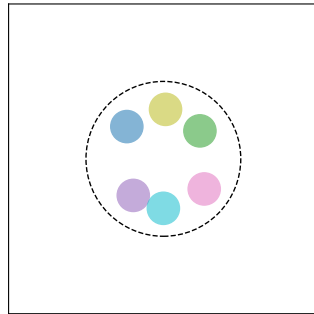
reconstruction alone:
blobs fly apart $\rightarrow$ holes



KL alone:
all pile on $\mathcal{N}(0, I) \rightarrow$ no info



balance (the VAE):
blobs tile $\mathcal{N}(0, I)$



Reconstruction alone spreads codes apart (sharp rebuilds, but the holes come back); the **KL** alone collapses them onto the prior (samplable, but they carry no information). The VAE keeps just enough of each.

The reparameterization trick

You cannot differentiate a coin flip. Unless.

Backprop through randomness

The ELBO needs a sample $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$.

But **sampling is random** - there is no gradient through a random node, so backprop stops dead.

Predict: we need a derivative through a random draw. How?

Backprop through randomness

The ELBO needs a sample $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$.

But **sampling is random** - there is no gradient through a random node, so backprop stops dead.

Predict: we need a derivative through a random draw. How?

The trick: move the randomness into an *input*.

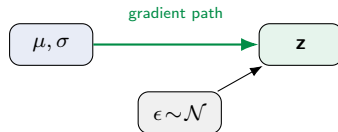
$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

Now $\boldsymbol{\mu}, \boldsymbol{\sigma}$ are deterministic - gradients flow through them; only $\boldsymbol{\epsilon}$ is random, and it carries no parameters.

before: gradient blocked



after: gradient flows



With this, the VAE trains end to end with plain SGD / Adam - the same optimizer from ch5.

The VAE in action

Sample the prior, decode, done.

Generate: sample the prior, decode

Training is done. To generate a brand-new digit that was never in the data:

1. draw $\mathbf{z} \sim \mathcal{N}(0, I)$,
2. decode $\hat{\mathbf{x}} = g(\mathbf{z})$.

That is it. Because the KL term made the codes fill $\mathcal{N}(0, I)$, almost every draw decodes to a valid digit.

This is the step the plain autoencoder could not take - its codes never matched any distribution you could sample.

VAE: draw $\mathbf{z} \sim \mathcal{N}(0, I)$, decode $\rightarrow$ new digits



The fix, side by side

Decode the *same* $\mathcal{N}(0, I)$ draws with each model:

Decode the same $z \sim \mathcal{N}(0, I)$: AE unreliable, VAE valid digits

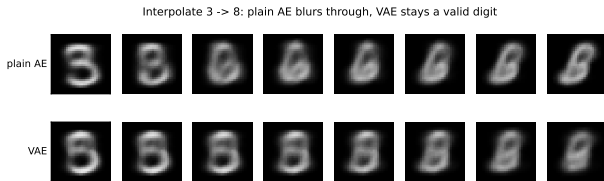


The plain AE collapses to repetitive, unreliable blobs (its codes do not live where you are sampling). The VAE gives **diverse, valid digits** - the KL prior did its job.

Walk the latent space

Because neighbouring codes decode to similar digits, you can **interpolate**: encode a 3 and an 8, walk the straight line between their codes, decode each step.

The VAE path stays closer to valid digits than the plain AE's (top row). And the full latent grid from the cold open shows the same thing everywhere: a smooth, organized space you can **traverse**.



One knob: β -VAE and disentanglement

Multiply the KL term by a weight β :

$$\mathcal{L} = \underbrace{\mathbb{E}_q[\log p(\mathbf{x} | \mathbf{z})]}_{\text{reconstruction}} - \beta \cdot \text{KL}(q(\mathbf{z} | \mathbf{x}) \| \mathcal{N}(0, I)).$$

- ▶ β large: press harder toward $\mathcal{N}(0, I)$. The latent axes tend to **disentangle** into interpretable factors - one axis becomes rotation, another thickness, another width.
- ▶ The price: blurrier reconstructions (you spent capacity on a tidy latent).

A single scalar trades **reconstruction quality** against **latent structure**. One example of a whole design space; the mechanics are still just the ELBO.

The VAE is inside modern image generators

You might think diffusion models replaced the VAE. The opposite is true - they run *inside* one.

- ▶ **Latent diffusion (Stable Diffusion).** Do not diffuse pixels - diffuse in a **VAE's latent**. The VAE compresses a $512 \times 512 \times 3$ image to $64 \times 64 \times 4$ (about $48\times$ fewer numbers); the diffusion model works in that latent; the **VAE decoder** paints the final image. Every Stable Diffusion or Midjourney image is decoded by a VAE.
- ▶ **Images as tokens (VQ-VAE).** Give the VAE a discrete codebook and an image becomes a **sequence of tokens** - so a transformer can generate images the same way an LLM generates text (the DALL-E lineage).

The VAE did not lose to diffusion. It became the **compression layer inside it**.

Honest limits and the road ahead

Principled, but blurry.

VAE samples are blurry

You have seen it on every VAE figure in this deck: the samples are recognizable but **soft**.

Why: the reconstruction loss rewards the **average** of all plausible outputs, and a Gaussian decoder smears detail. A VAE gives you a **principled latent space and a likelihood**, but not sharp images.

VAE	principled latent + likelihood; blurry
GAN	sharp images; no likelihood, unstable (<i>L19 showcase</i>)
Diffusion	state of the art - and it runs <i>inside</i> a VAE latent

Recap and the road ahead

- ▶ A VAE is an autoencoder whose encoder outputs a **distribution** per input, trained to fill $\mathcal{N}(0, I)$.
- ▶ Its loss is the **ELBO** = reconstruction – KL; the KL is a prior that removes the holes.
- ▶ The **reparameterization trick** makes the random sampling differentiable, so it trains with plain gradient descent.
- ▶ Sample the prior to **generate**; samples are valid but blurry.

Next - the GenAI chapter. Two roads meet there: the **attention** thread from L21, and this generative thread. Ahead: transformers, GANs, diffusion, and LLMs.